Corporate Equipment Allocation and Tracking System

1. Introduction

The Corporate Equipment Allocation and Tracking System is designed to streamline the distribution and tracking of office equipment, such as laptops, projectors, and tablets. Reception staff can efficiently allocate equipment, track its availability, and ensure timely returns. The system also automates notifications to employees, maintenance, and inventory teams, improving operational efficiency and accountability.

2. Problem Statement

Managing office equipment manually can lead to delays, misplaced items, and missed maintenance requirements. Reception staff need a system to:

- Record equipment allocations and returns.
- Track employee usage and expected return dates.
- Automate reminders for overdue returns.
- Alert maintenance for damaged equipment.
- Keep inventory records up-to-date.

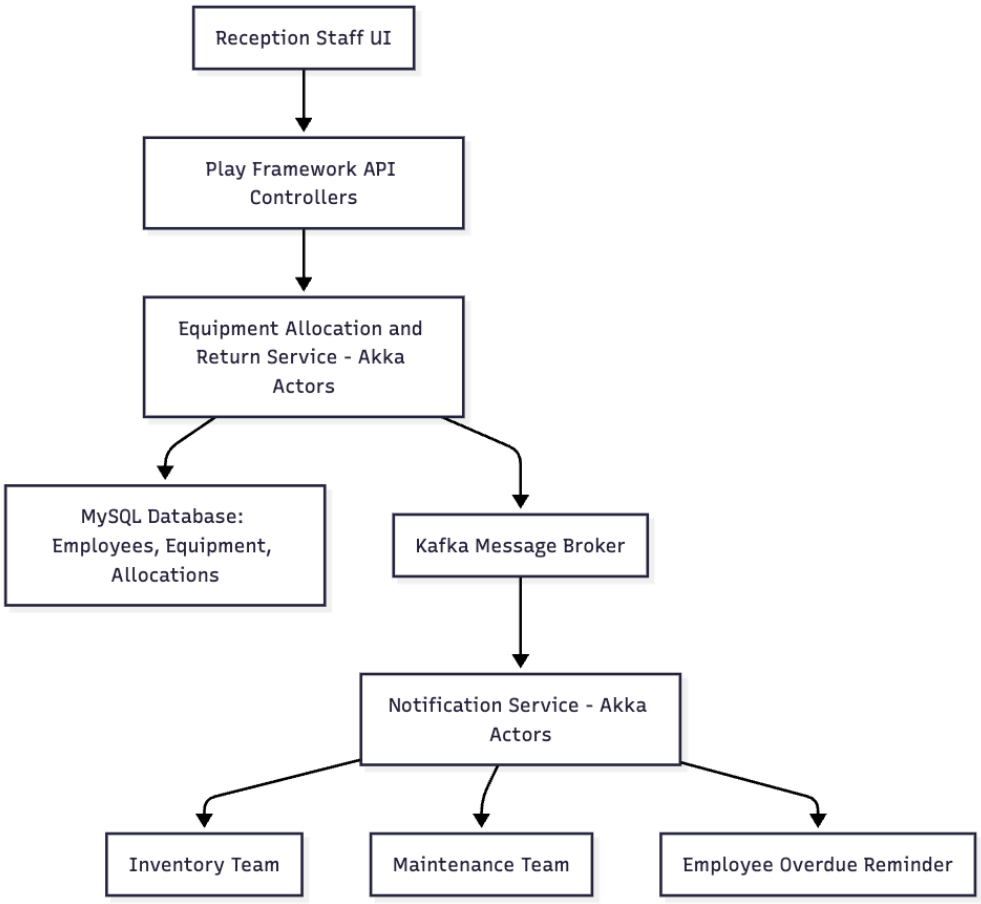
This system aims to solve these challenges by providing a reliable, scalable, and automated solution.

3. Architecture Overview

The system follows a **modular microservice-based architecture**. The key components include:

1. **Frontend / Application Layer** – Interfaces for reception staff to request allocation/return and view equipment status.
2. **API Layer (Play Framework)** – Handles REST requests for allocation, return, and status updates.
3. **Service Layer** – Contains business logic, validation, and workflows.
4. **Database Layer (MySQL)** – Stores employees, equipment, and allocation records.

5. **Notification Service (Akka Actors)** – Handles asynchronous notifications like overdue reminders and maintenance alerts.
6. **Message Broker (Kafka)** – Queues notifications to ensure reliable delivery.
7. **Deployment** – Docker containers for consistent and scalable deployment.



4. Component Design

Frontend / App:

- User interface for reception staff.
- Submit allocation and return requests.
- View status of equipment.

Play API Controllers:

- Receive and validate REST API requests.
- Pass data to the Service Layer.

Service Layer:

- Apply business logic for allocation and return.
- Trigger notifications based on events (overdue, damage, etc.).

Database (MySQL):

- Tables: Employees, Equipment, Allocations.
- Stores allocation history and status.

Akka Notification Service:

- Actors manage asynchronous notifications.
- Schedulers handle reminders for overdue equipment.
- Kafka consumers listen for events from the database/service layer.

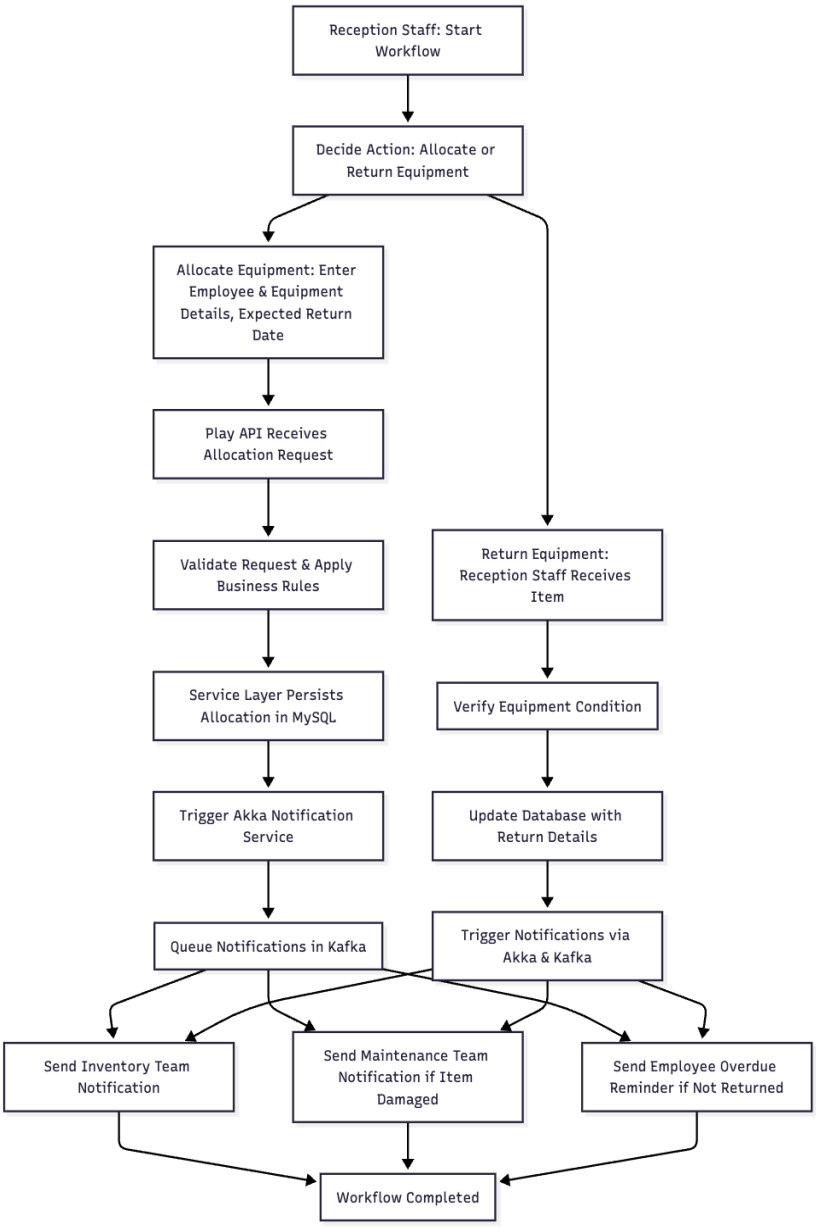
Kafka (Messaging Layer):

- Queue notifications for delivery to email services.
- Ensures no notifications are lost even if services are busy.

Docker Deployment:

- Each component runs in its own container for consistency.
- Supports scalability and easy environment replication.

5. System Workflow



6. Microservices Design

The system implements a microservice architecture for modularity and scalability.

Microservices and Purpose

- Equipment Allocation Service:
  - Handles allocation, return, and equipment status tracking.
  - Updates the database with allocation and return information.
  - Applies business rules to ensure equipment availability.
- Notification Service:
  - Sends reminders to employees for overdue equipment.
  - Alerts maintenance for damaged items.
  - Notifies inventory about allocations and returns.

Inter-Service Communication

- Synchronous: Frontend → Equipment Allocation Service via REST API.
- Asynchronous: Equipment Allocation Service → Notification Service via Kafka.

Design Rationale:

- Separating notification logic from allocation ensures high responsiveness.
- Independent scaling is possible for services under heavy load.
- Improves maintainability and modularity.

7. REST API Endpoints

The Corporate Equipment Allocation and Tracking System exposes the following REST APIs to manage equipment allocation, return, and status tracking. These endpoints are primarily consumed by the frontend application used by reception staff.

7.1 API Request & Response Examples

Authentication

1. **Login**

URL: POST — http://localhost:9000/api/login

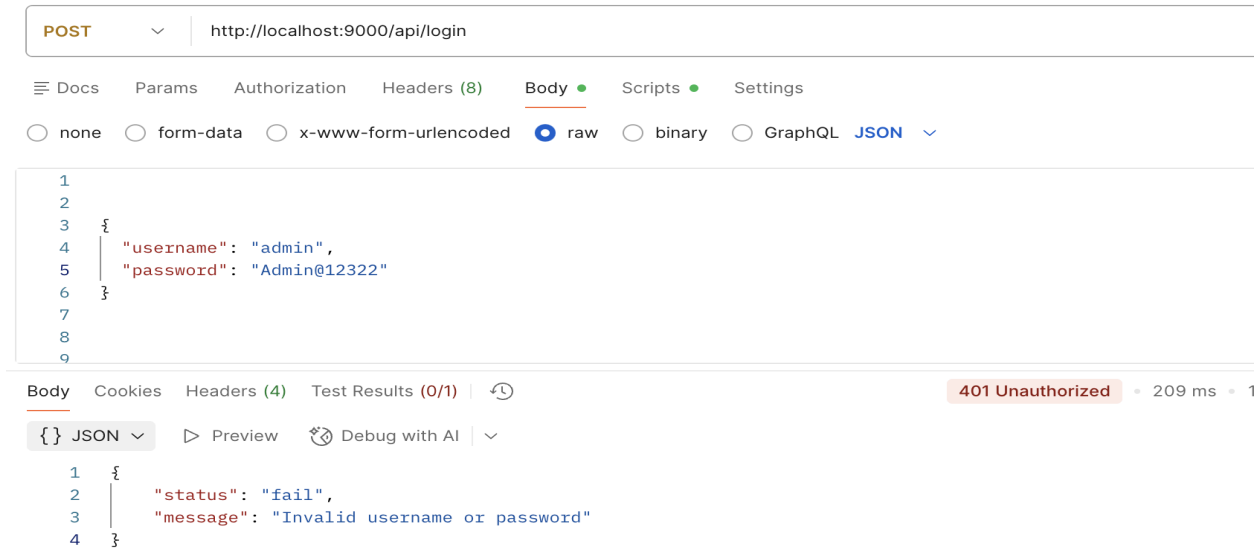
Input:

```
{
 "username": "admin",
 "password": "Admin@123"
}
```

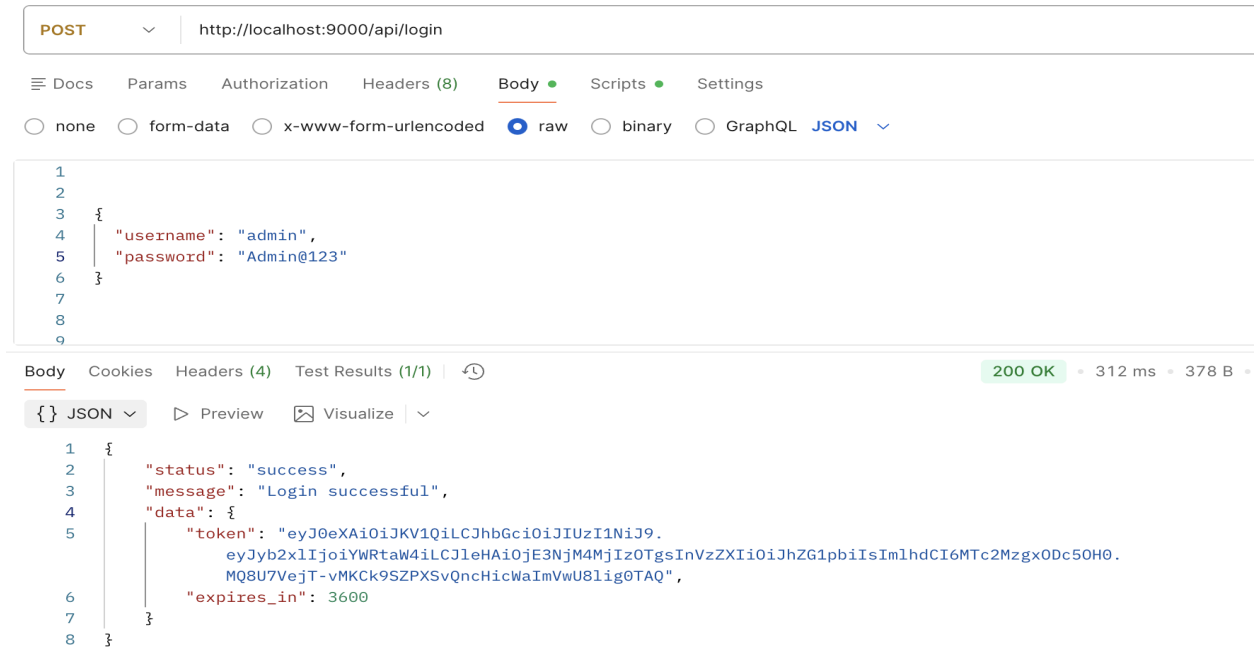
Output:


```
{
   "status": "success",
   "message": "Login successful",
   "data": {
       "token":
"eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJyb2xlIjoiYWRtaW4iLCJleHAiOjE3NjM4MjY0M
zMsInVzZXIiOiJhZG1pbiIsImlhdCI6MTc2MzgyMjgzM30.YwsJ5pb0BVWPtsra7XdgFoju8RJ8iMri
h8_zzPsbiWA",
       "expires_in": 3600
   }
}
```

Case fail:



Case Success:



User Management

Create User

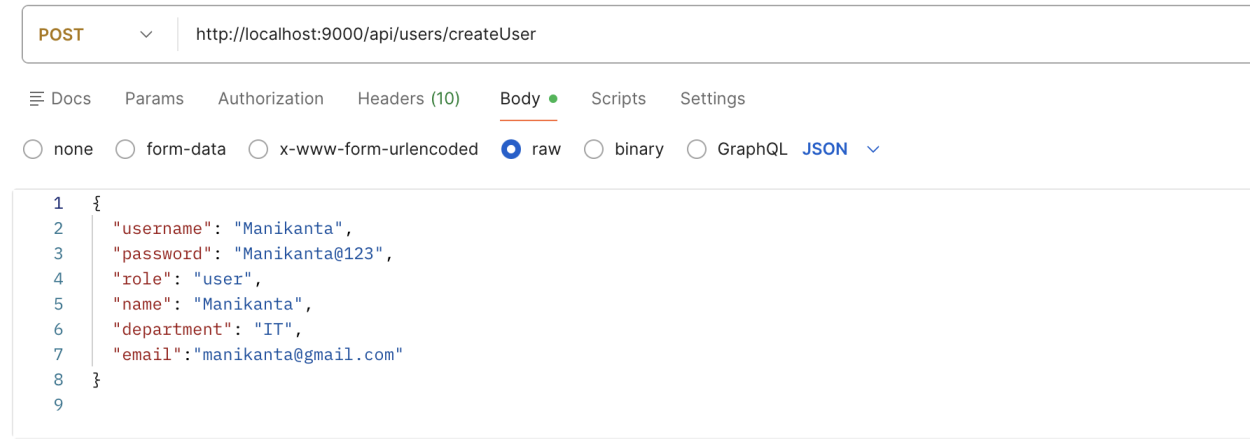
URL: POST http://localhost:9000/api/users/createUser

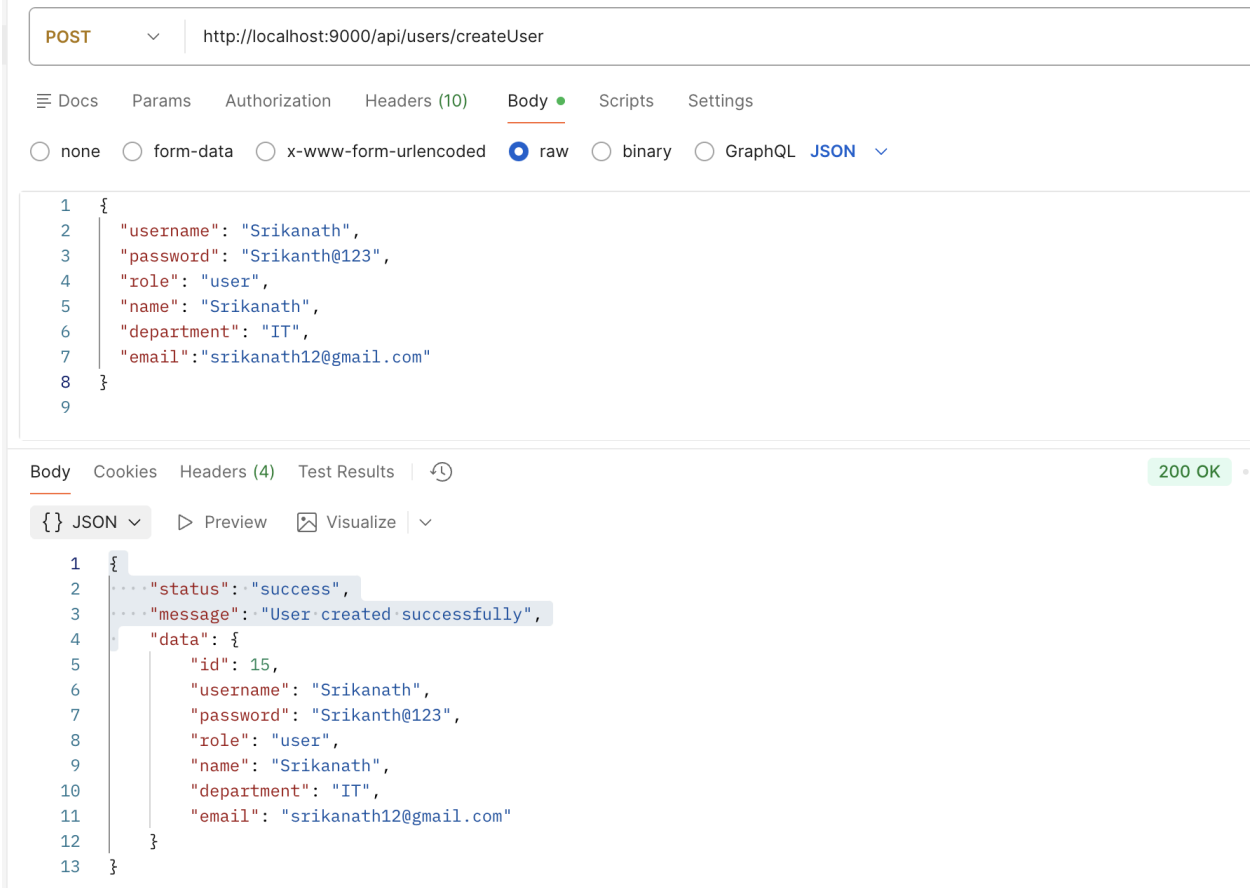
Input:


```
{
 "username": "Manikanta",
 "password": "Manikanta@123",
 "role": "user",
 "name": "Manikanta",
 "department": "IT",
 "email":"manikanta@gmail.com"
}
```


OUTPUTS:

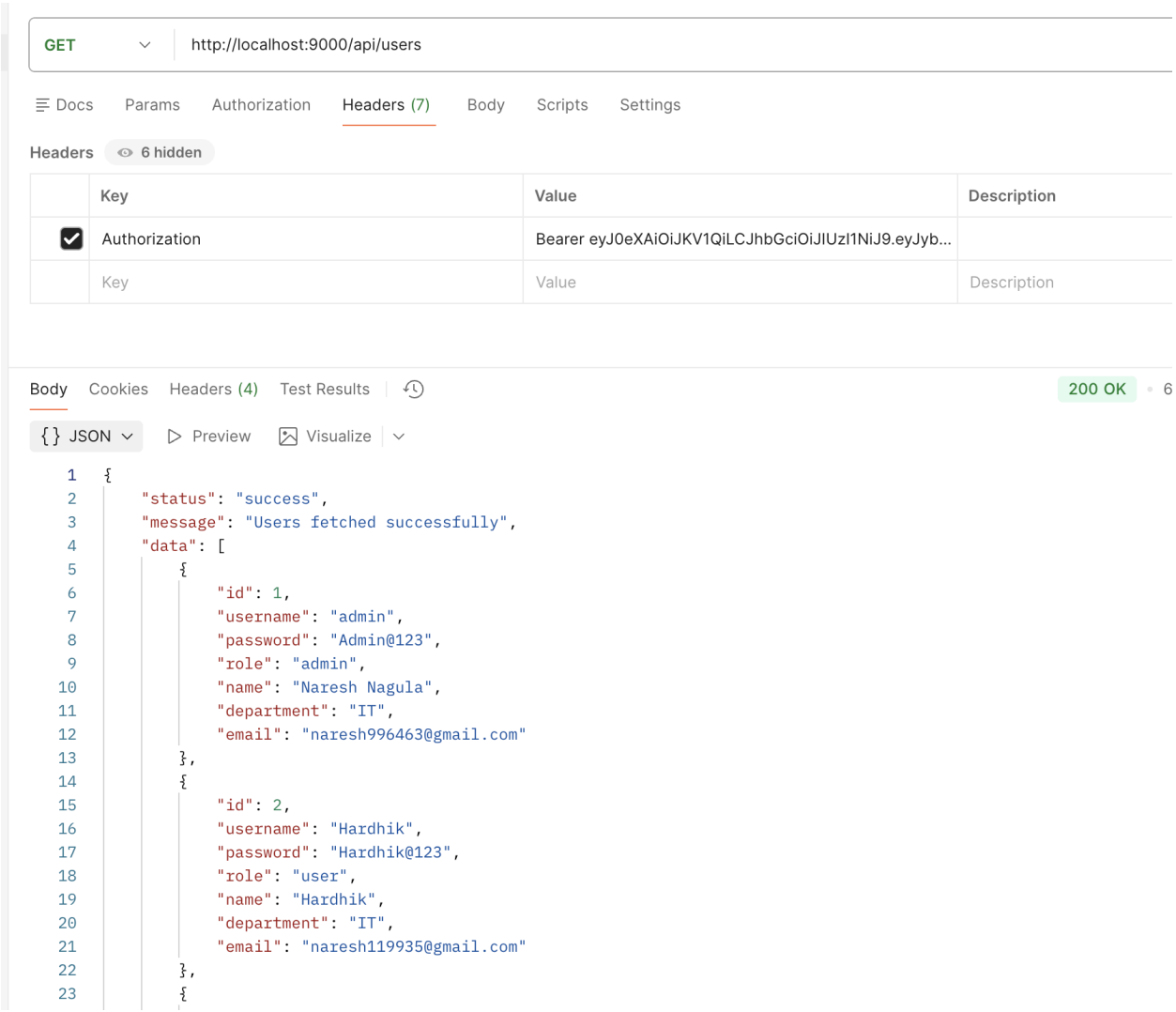

```
{
   "status": "fail",
   "message": "Username 'Manikanta' already exists"
}
{
   "status": "success",
   "message": "User created successfully",
   "data": {
       "id": 15,
       "username": "Srikanath",
       "password": "Srikanth@123",
       "role": "user",
       "name": "Srikanath",
       "department": "IT",
       "email": "srikanath12@gmail.com"
   }
}
```


POST
http://localhost:9000/api/users/createUser
Docs
Params
Authorization
Headers (10)
Body
Scripts
Settings
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
{
"username": "Manikanta",
"password": "Manikanta@123",
"role": "user",
"name": "Manikanta",
"department": "IT",
"email":"manikanta@gmail.com"
}
Body
Cookies
Headers (4)
Test Results
200 OK
JSON
Preview
Visualize
{
"status": "fail",
"message": "Username 'Manikanta' already exists"
}


POST
http://localhost:9000/api/users/createUser
Docs
Params
Authorization
Headers (10)
Body
Scripts
Settings
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
{
"username": "Srikanath",
"password": "Srikanth@123",
"role": "user",
"name": "Srikanath",
"department": "IT",
"email":"srikanath12@gmail.com"
}
Body
Cookies
Headers (4)
Test Results
200 OK
JSON
Preview
Visualize
{
"status": "success",
"message": "User created successfully",
"data": {
"id": 15,
"username": "Srikanath",
"password": "Srikanth@123",
"role": "user",
"name": "Srikanath",
"department": "IT",
"email": "srikanath12@gmail.com"
}
}

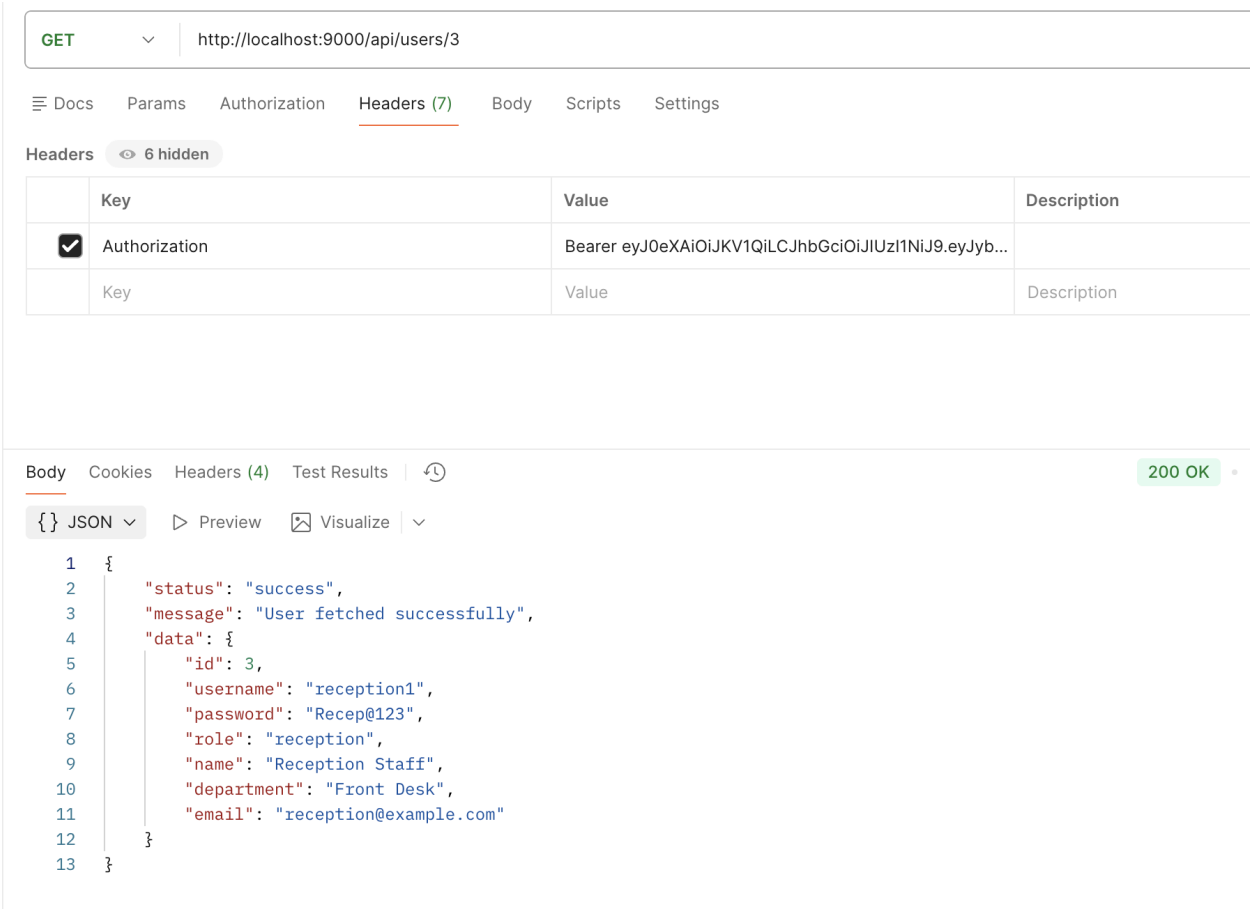
Get All Users

URL: GET http://localhost:9000/api/users



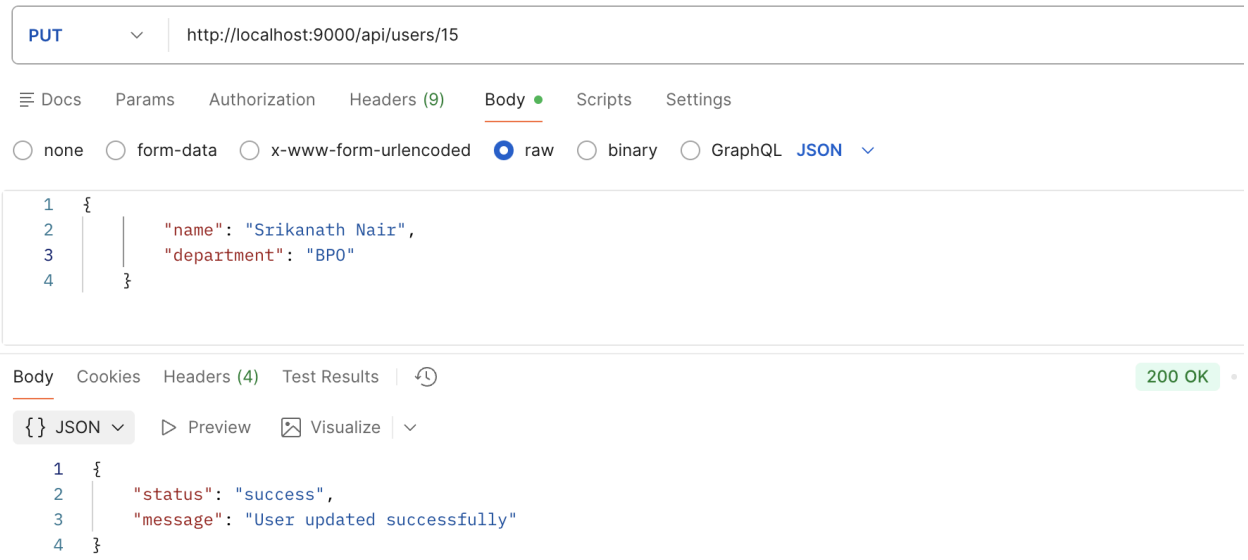
Get User by ID

URL: GET http://localhost:9000/api/users/3



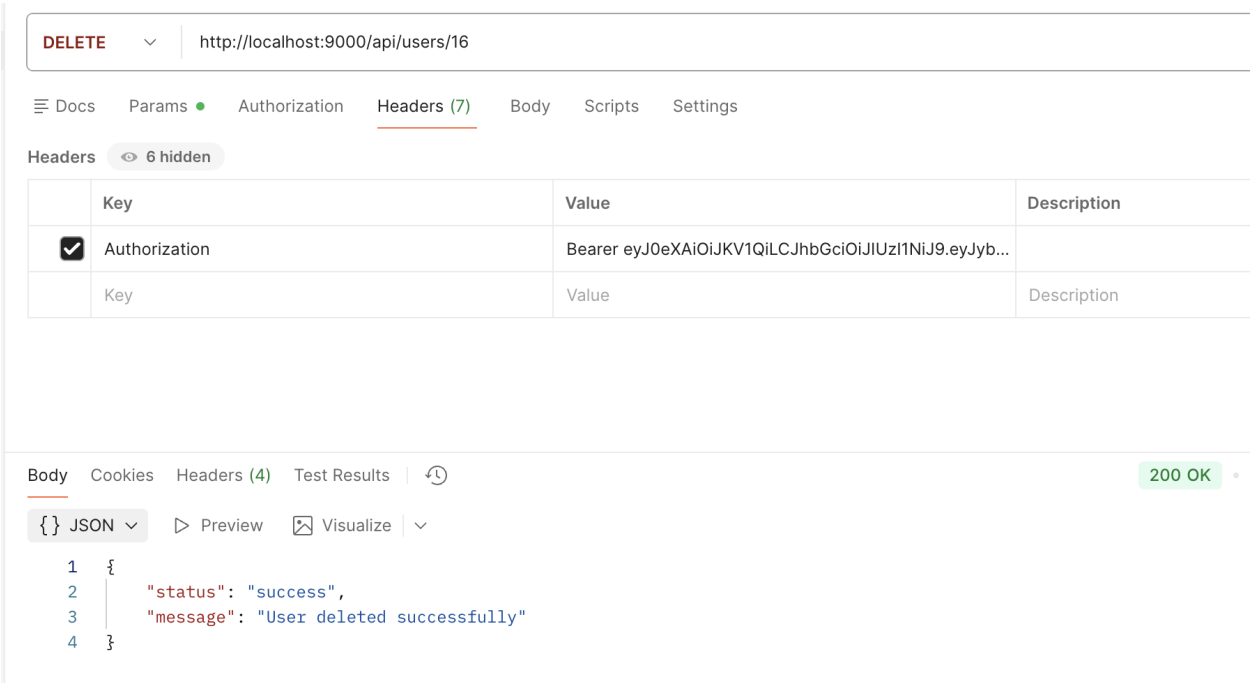
Update User

URL: PUT http://localhost:9000/api/users/15



Delete User

URL: PUT http://localhost:9000/api/users/15



Equipment Management

Create Equipment

- URL: POST — http://localhost:9000/api/equipment/create
- Purpose: Created equipment details

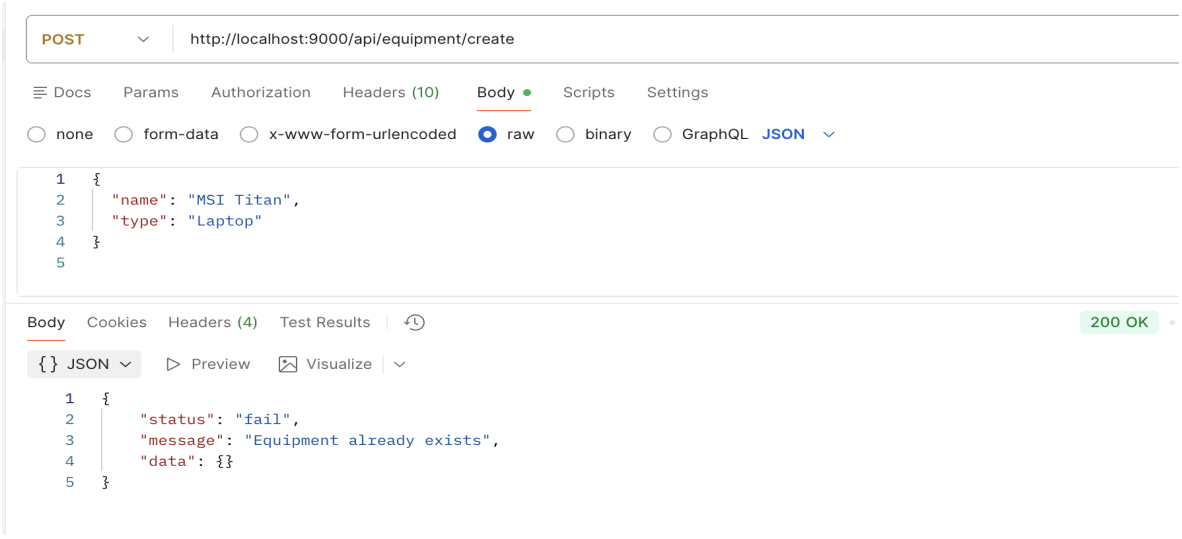
Input:

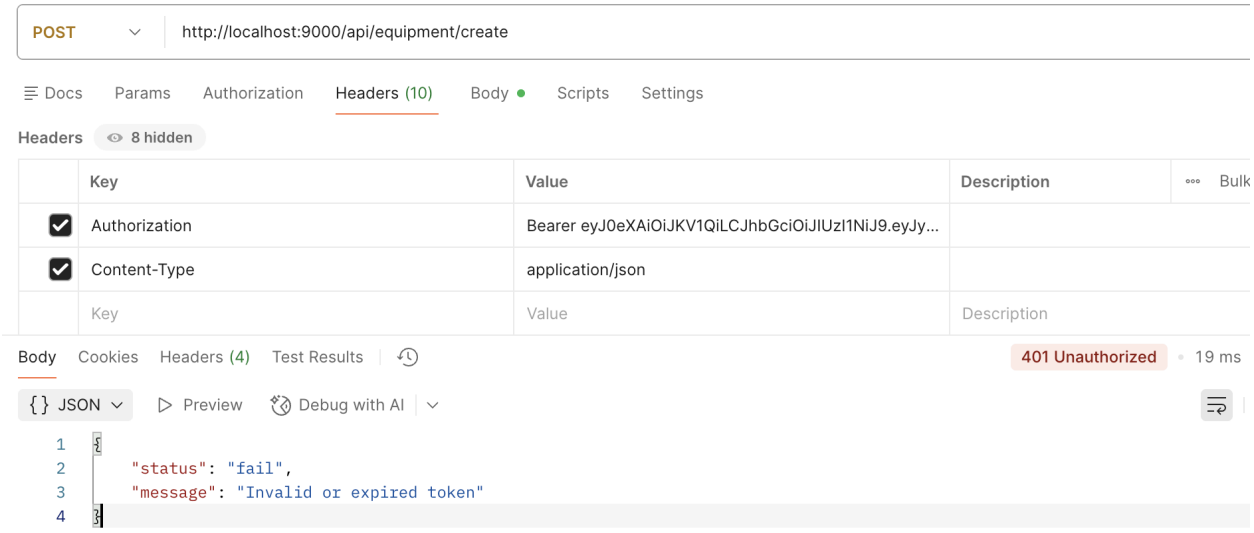

```
{
 "name": "MSI Titan",
 "type": "Laptop"
}
```


Output

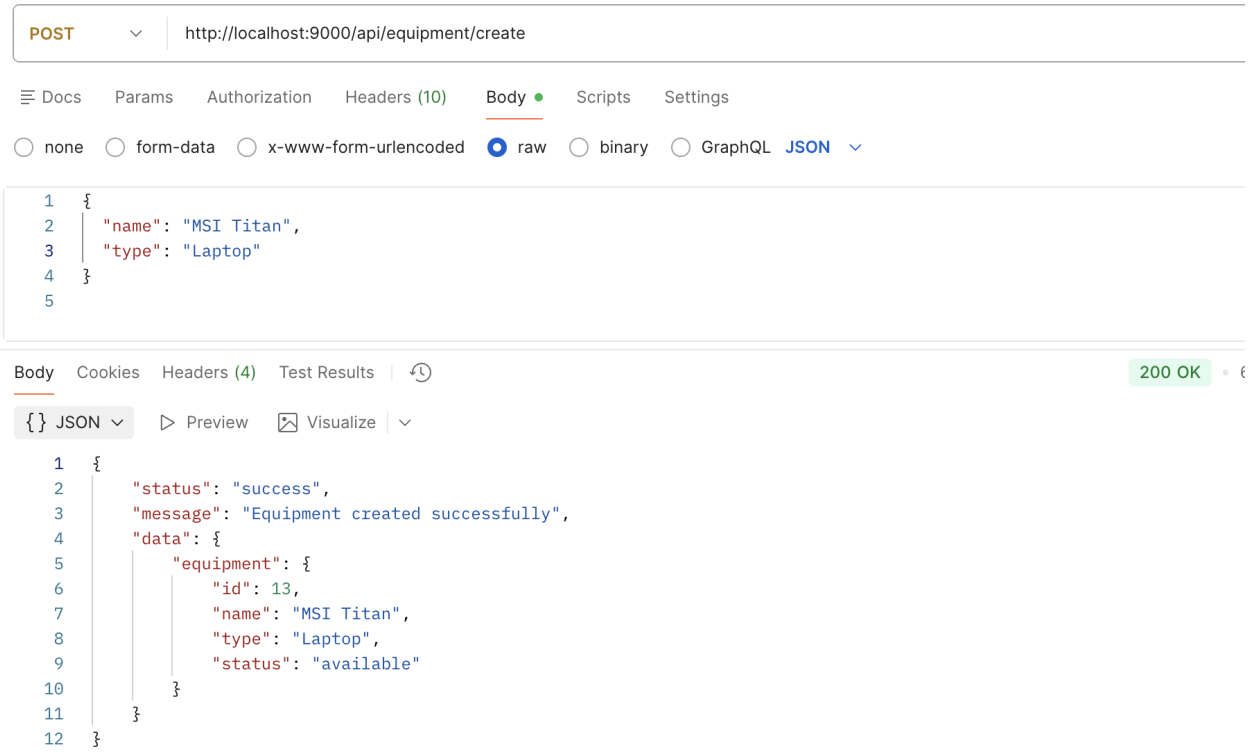

```
{
   "status": "success",
   "message": "Equipment created successfully",
   "data": {
       "equipment": {
           "id": 14,
           "name": "MSI Titan Xpro",
           "type": "Laptop",
           "status": "available"
       }
   }
}
```

Case fail:


POST
http://localhost:9000/api/equipment/create
Docs
Params
Authorization
Headers (10)
Body
Scripts
Settings
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
{
"name": "MSI Titan",
"type": "Laptop"
}
Body
Cookies
Headers (4)
Test Results
200 OK
{} JSON
Preview
Visualize
{
"status": "fail",
"message": "Equipment already exists",
"data": {}
}


POST
http://localhost:9000/api/equipment/create
Docs
Params
Authorization
Headers (10)
Body
Scripts
Settings
Headers
8 hidden
Key
Value
Description
Bulk
Authorization
Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJy...
Content-Type
application/json
Key
Value
Description
Body
Cookies
Headers (4)
Test Results
401 Unauthorized
19 ms
{} JSON
Preview
Debug with AI
{
"status": "fail",
"message": "Invalid or expired token"
}

Case Success:



Update Equipment

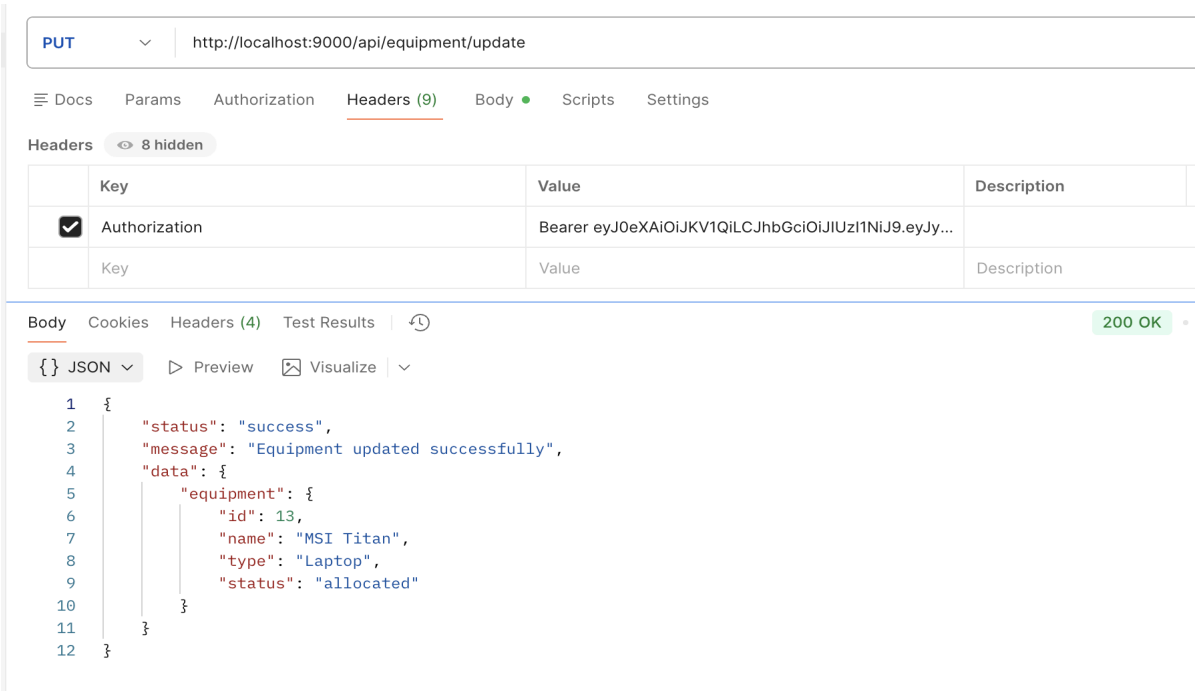
URL: PUT — http://localhost:9000/api/equipment/update

Input:

```
{
"id": 13,
"name": "MSI Titan",
"type": "Laptop",
"status": "allocated"
}
```

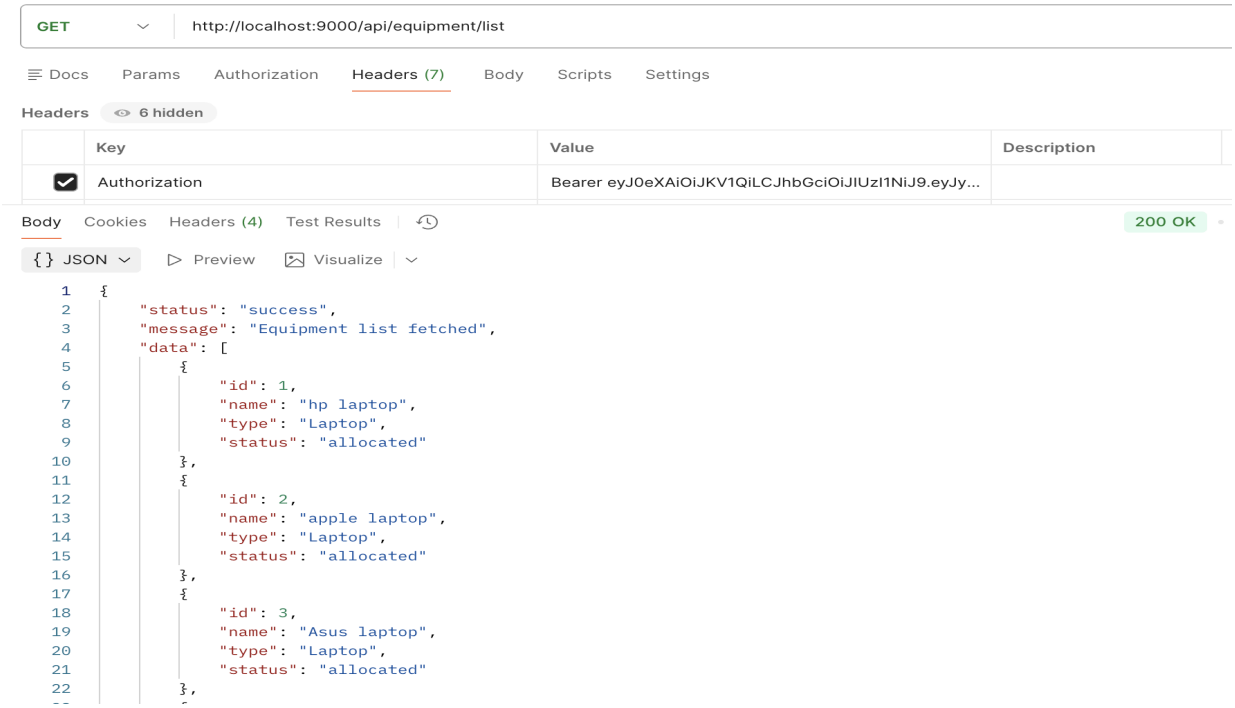
Output:


```
{
    "status": "success",
    "message": "Equipment updated successfully",
    "data": {
        "equipment": {
            "id": 13,
            "name": "MSI Titan",
            "type": "Laptop",
            "status": "allocated"
        }
    }
}
```

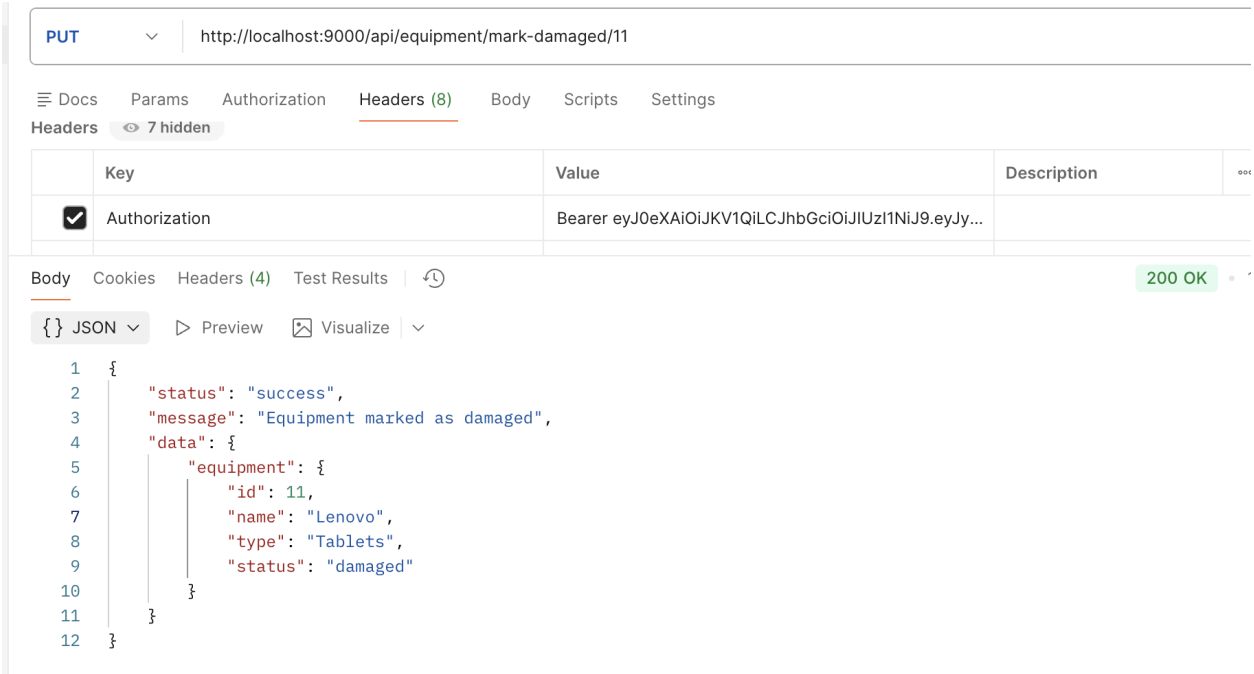
List Equipment:

URL: GET http://localhost:9000/api/equipment/list



Mark Damaged Equipment

URL: PUT http://localhost:9000/api/equipment/mark-damaged/:equipmentId



Allocation and Return APIs

Allocate Equipment

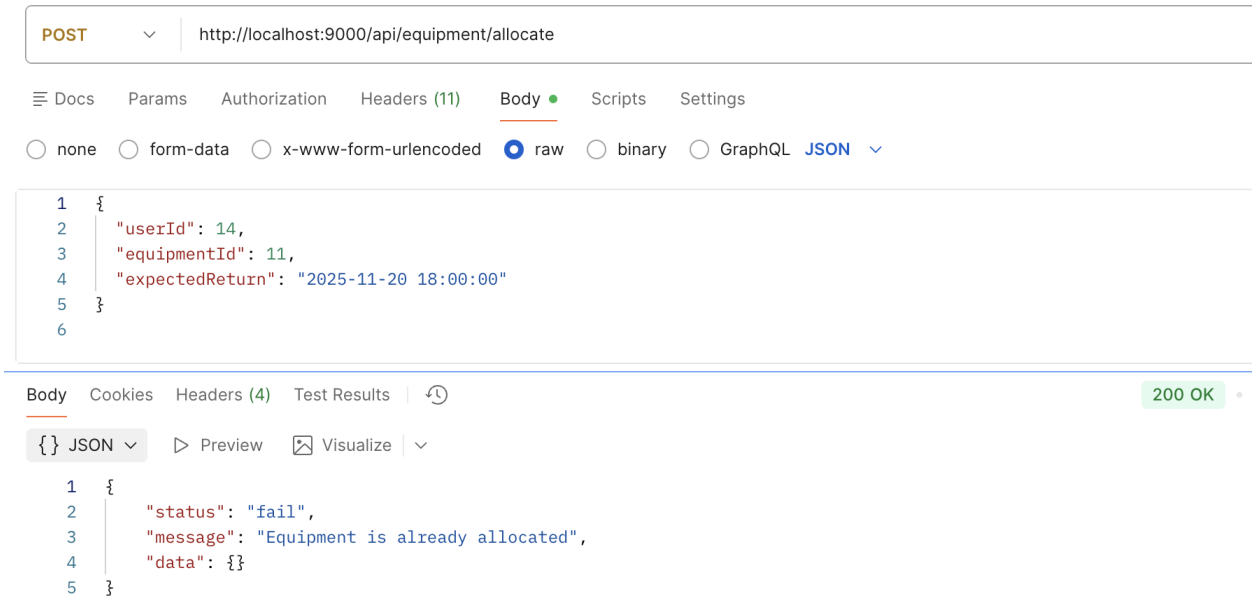
URL: POST http://localhost:9000/api/equipment/allocate

Behaviour:

Success:
Creates allocation. Equipment marked allocated. Kafka event sent.
Fail:
Invalid body, equipment already allocated, or missing users.
Input:

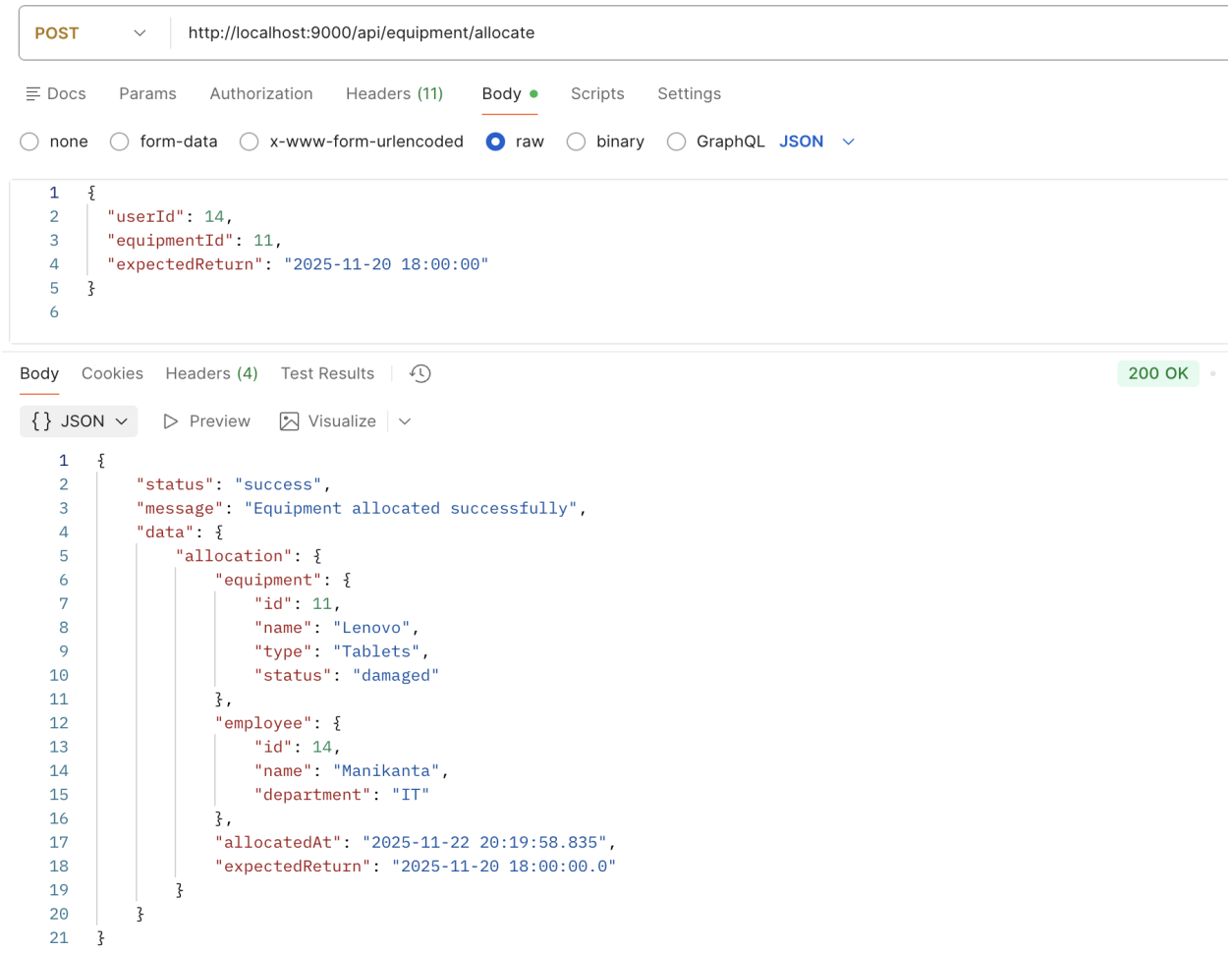
```
{
 "userId": 14,
 "equipmentId": 11,
 "expectedReturn": "2025-11-20 18:00:00"
}
```

Case Fail:



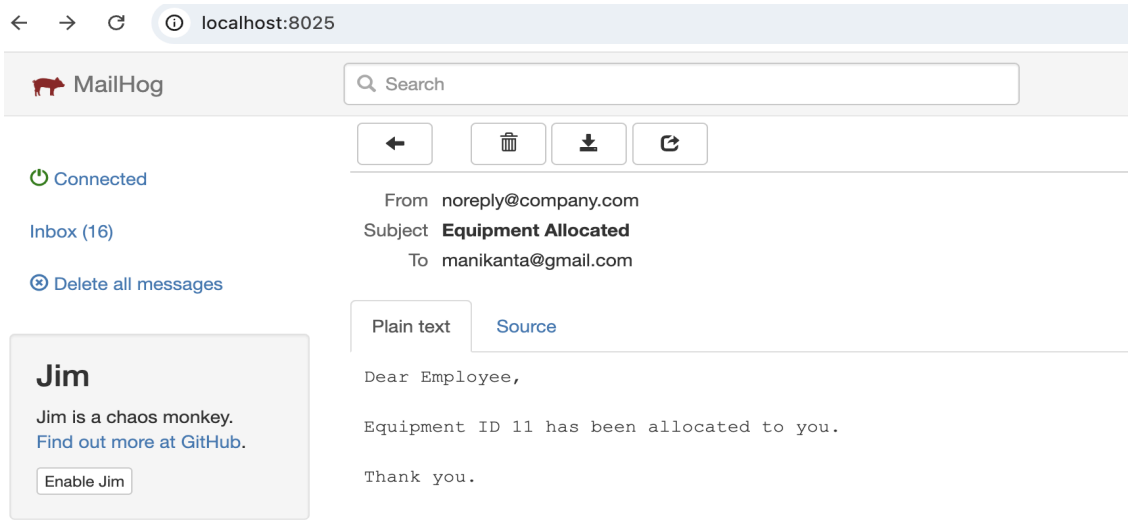
Case Success:


```
{
   "status": "success",
   "message": "Equipment allocated successfully",
   "data": {
       "allocation": {
           "equipment": {
               "id": 11,
               "name": "Lenovo",
               "type": "Tablets",
               "status": "allocated"
           },
           "employee": {
               "id": 14,
               "name": "Manikanta",
               "department": "IT"
           },
           "allocatedAt": "2025-11-22 20:22:37.347",
           "expectedReturn": "2025-11-20 18:00:00.0"
       }
   }
}
```

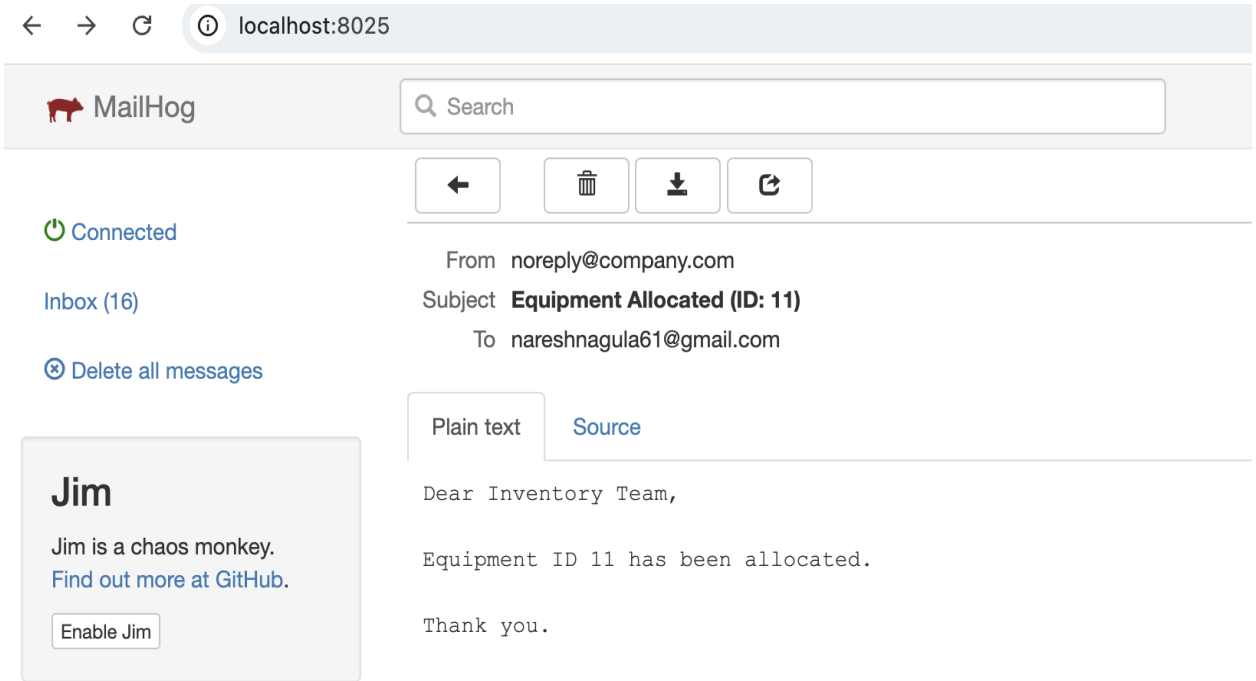

POST
http://localhost:9000/api/equipment/allocate
Docs
Params
Authorization
Headers (11)
Body
Scripts
Settings
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
{
"userId": 14,
"equipmentId": 11,
"expectedReturn": "2025-11-20 18:00:00"
}
Body
Cookies
Headers (4)
Test Results
200 OK
JSON
Preview
Visualize
{
"status": "success",
"message": "Equipment allocated successfully",
"data": {
"allocation": {
"equipment": {
"id": 11,
"name": "Lenovo",
"type": "Tablets",
"status": "damaged"
},
"employee": {
"id": 14,
"name": "Manikanta",
"department": "IT"
},
"allocatedAt": "2025-11-22 20:19:58.835",
"expectedReturn": "2025-11-20 18:00:00.0"
}
}
}

Kafka notification triggered post-allocation for employee and inventory teams.

Employee



Inventory teams



Return Equipment

URL: POST http://localhost:9000/api/equipment/return

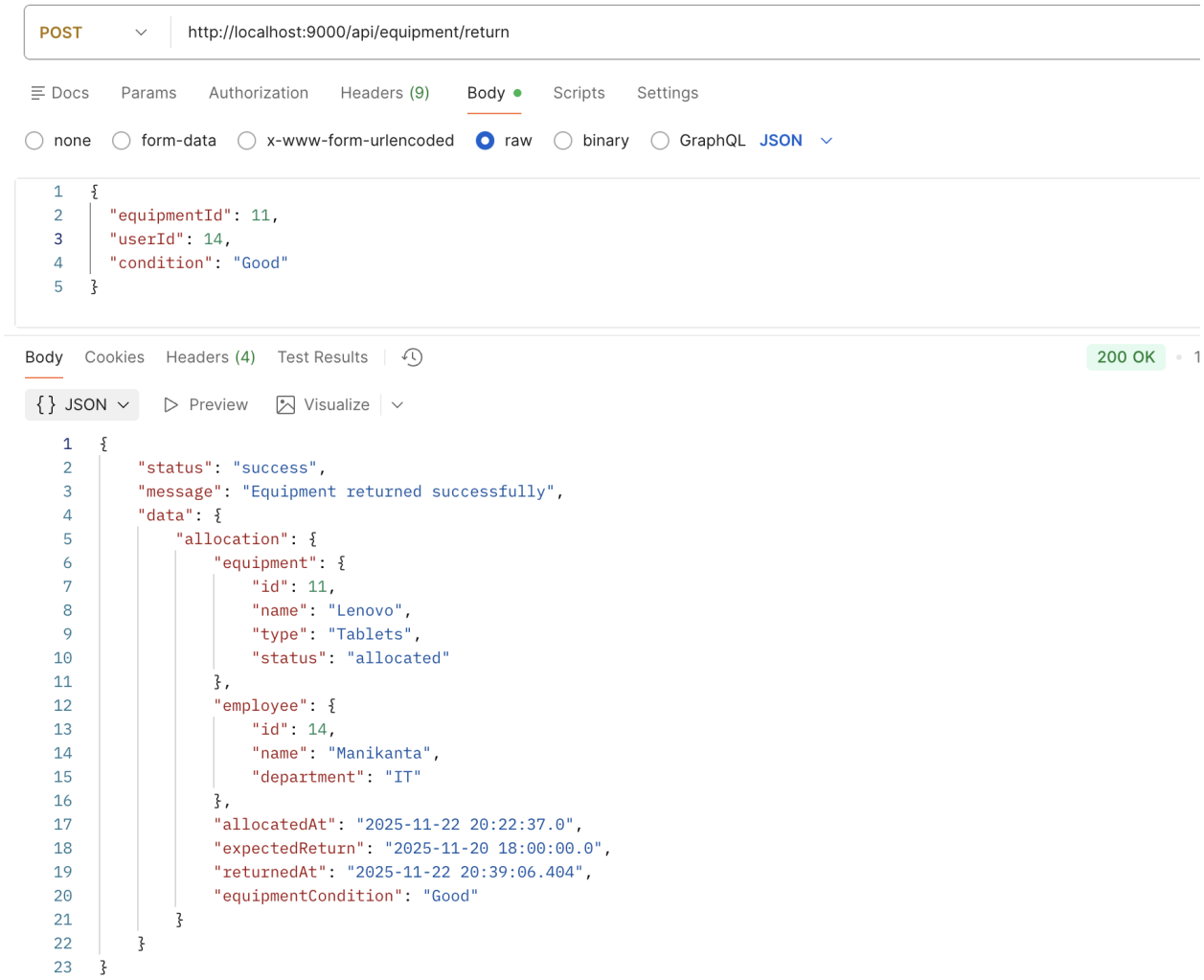
Behaviour:

Success:

Updates allocation. Equipment status updated. Kafka events triggered.

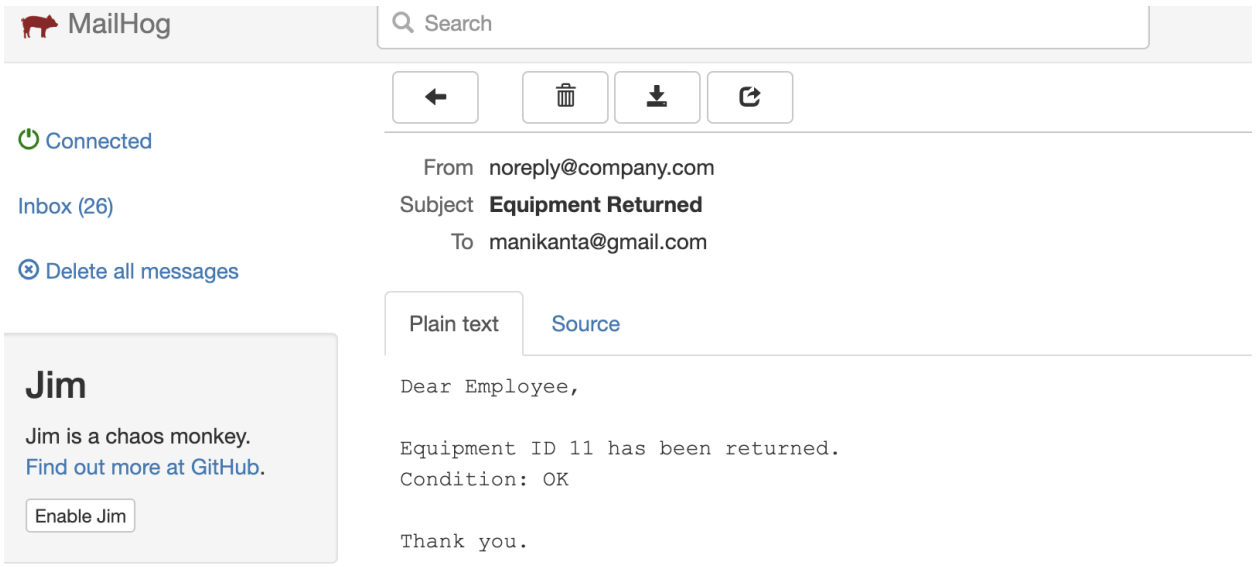
Fail:

Invalid body, no active allocation, or missing users.

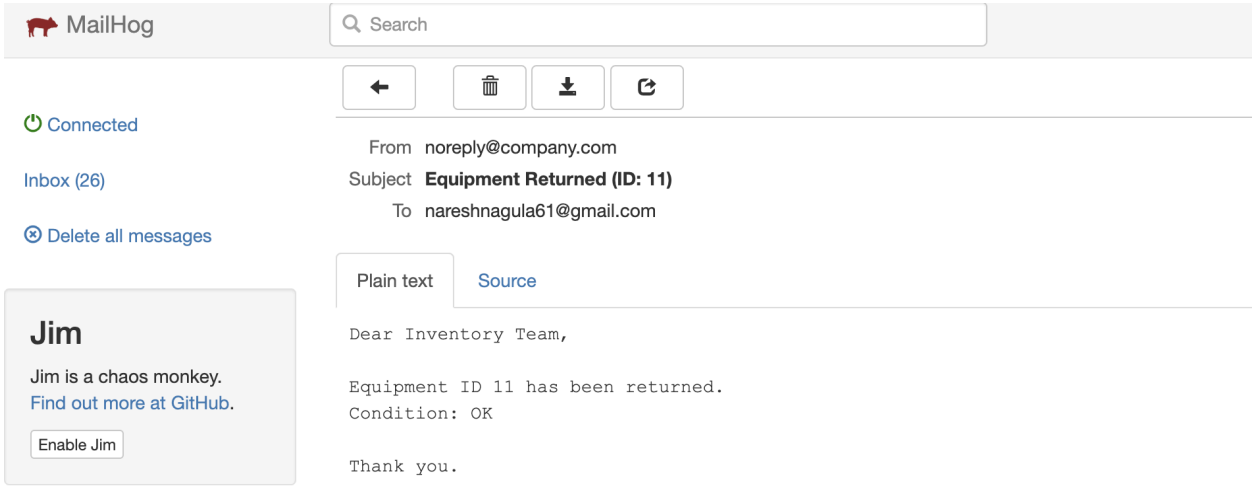


Kafka notification triggered post-allocation for employee and inventory teams.

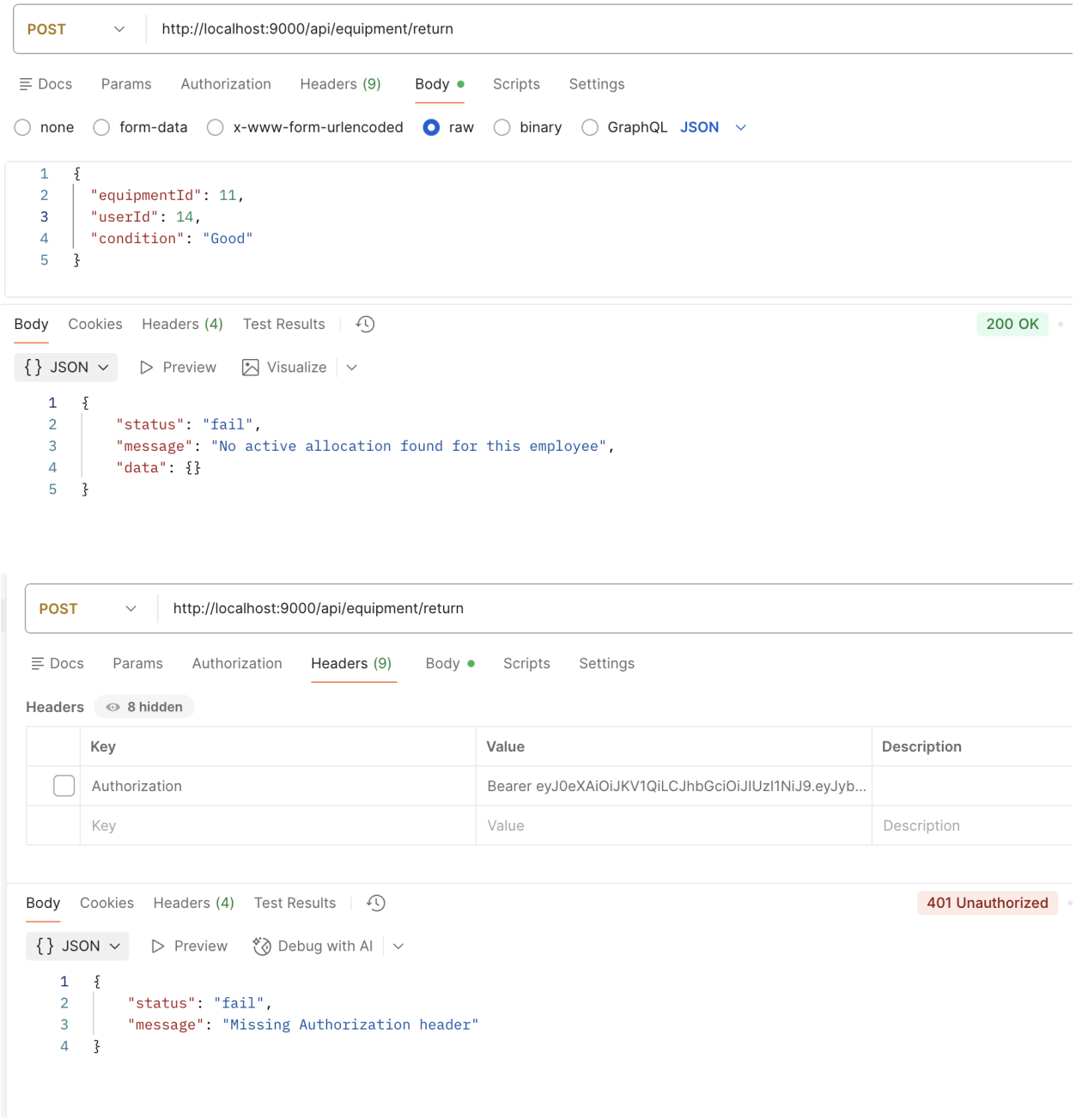
employee:



Inventory teams



Fail case:


POST
http://localhost:9000/api/equipment/return
Docs
Params
Authorization
Headers (9)
Body
Scripts
Settings
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
{
"equipmentId": 11,
"userId": 14,
"condition": "Good"
}
Body
Cookies
Headers (4)
Test Results
200 OK
JSON
Preview
Visualize
{
"status": "fail",
"message": "No active allocation found for this employee",
"data": {}
}
POST
http://localhost:9000/api/equipment/return
Docs
Params
Authorization
Headers (9)
Body
Scripts
Settings
Headers
8 hidden
Key
Value
Description
Authorization
Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJyb...
Key
Value
Description
Body
Cookies
Headers (4)
Test Results
401 Unauthorized
JSON
Preview
Debug with AI
{
"status": "fail",
"message": "Missing Authorization header"
}

Get Allocation Status by User

URL: GET http://localhost:9000/api/equipment/status/:userId

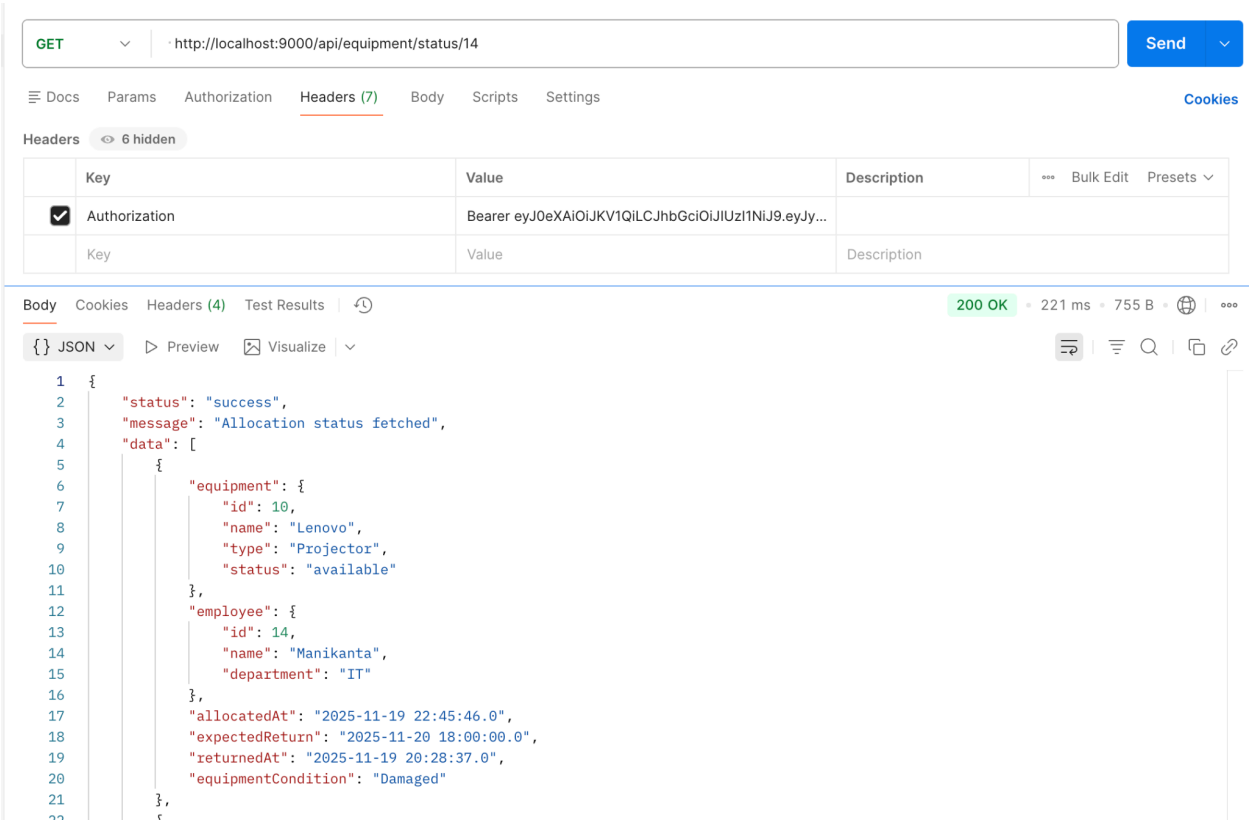
Behaviour:

Success:

Returns allocation list or empty list.

Fail:

Unauthorized.



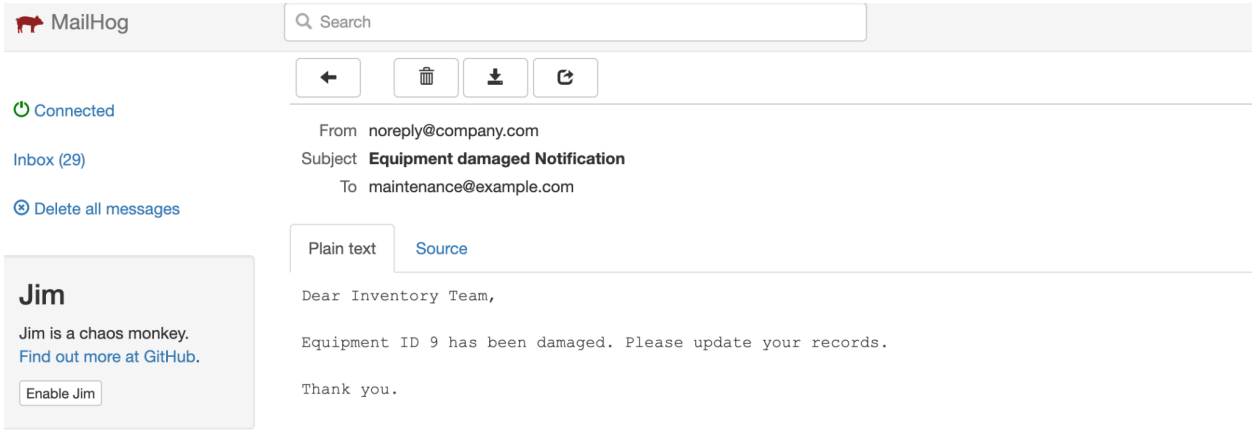
7.2 Notification Service

- **Purpose:** Handle all automated notifications related to equipment allocation and return.
- **Endpoints:** No public REST endpoints.
- **Behavior:**

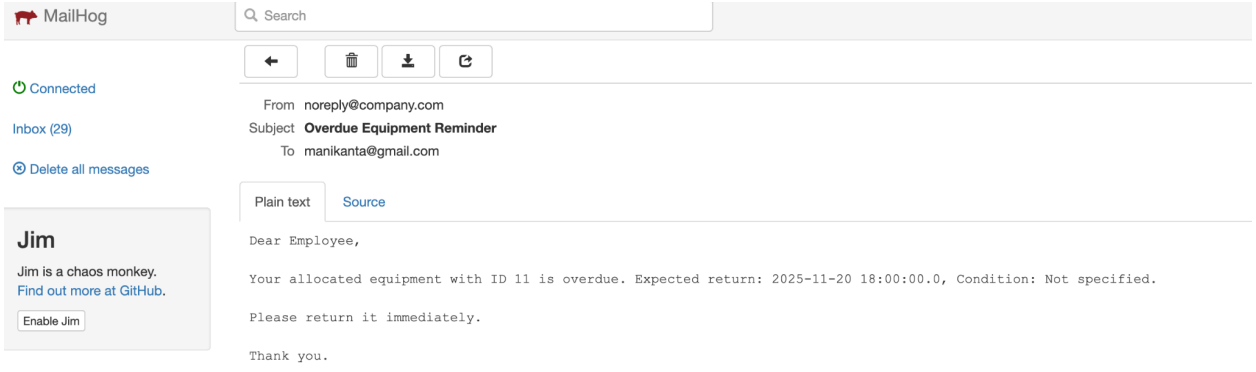
- Consumes events from Kafka triggered by allocation or return actions.
- Sends notifications via email/SMS to employees, inventory, and maintenance teams.

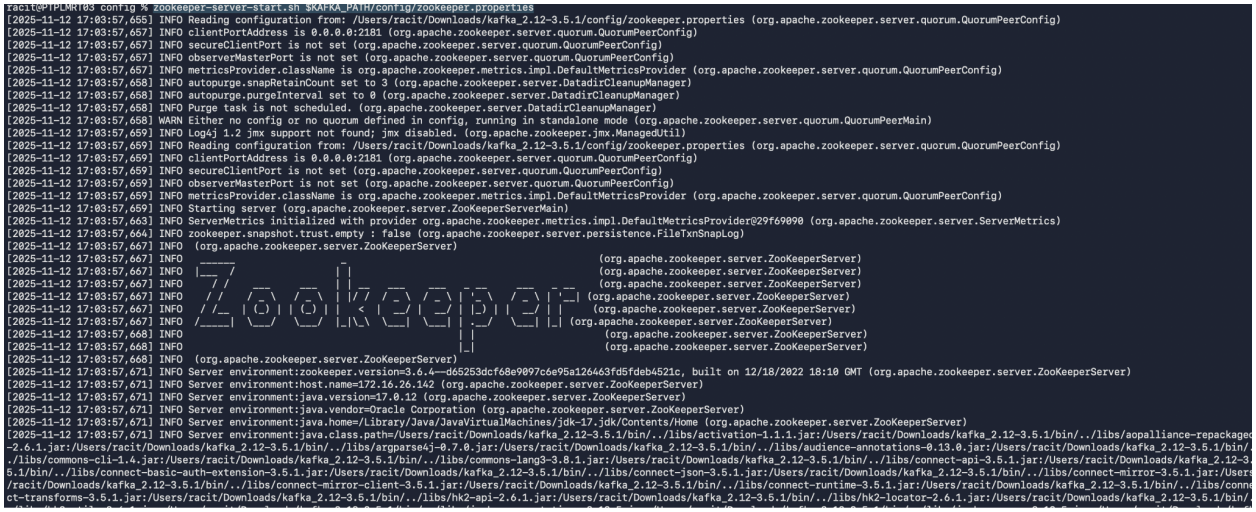
Ensures asynchronous, reliable delivery of reminders and alerts.

Notification sent for damaged equipment



Notification sent for overdue reminder






```
{"eventType":"overdue","allocationId":5,"equipmentId":5,"userId":9,"userEmail":"mahesh2@gmail.com","expectedReturn":"2025-11-20 18:00:00.0","notifiedAt":"2025-11-22 14:36:57.468"}
{"eventType":"overdue","allocationId":23,"equipmentId":11,"userId":14,"userEmail":"manikanta@gmail.com","expectedReturn":"2025-11-20 18:00:00.0","notifiedAt":"2025-11-22 14:36:57.474"}
{"eventType":"overdue","allocationId":9,"equipmentId":9,"userId":13,"userEmail":"srichaitra@gmail.com","expectedReturn":"2025-11-20 18:00:00.0","notifiedAt":"2025-11-22 14:36:57.471"}
{
"eventType": "damaged",
"equipmentId": 11,
"employeeEmail": "maintenance@example.com",
"inventoryEmail": "nareshnagula61@gmail.com"
}
{
"eventType": "damaged",
"equipmentId": 11,
"employeeEmail": "maintenance@example.com",
"inventoryEmail": "nareshnagula61@gmail.com"
}
{
"eventType": "allocated",
"equipmentId": 11,
"employeeEmail": "manikanta@gmail.com",
"inventoryEmail": "nareshnagula61@gmail.com"
}
{"eventType":"overdue","allocationId":24,"equipmentId":11,"userId":14,"userEmail":"manikanta@gmail.com","expectedReturn":"2025-11-20 18:00:00.0","notifiedAt":"2025-11-22 20:20:00.818"}
{
"eventType": "allocated",
"equipmentId": 11,
"employeeEmail": "manikanta@gmail.com",
"inventoryEmail": "nareshnagula61@gmail.com"
}
{"eventType":"overdue","allocationId":25,"equipmentId":11,"userId":14,"userEmail":"manikanta@gmail.com","expectedReturn":"2025-11-20 18:00:00.0","notifiedAt":"2025-11-22 20:22:40.765"}
{
"eventType": "returned",
"equipmentId": 11,
"employeeEmail": "manikanta@gmail.com",
"inventoryEmail": "nareshnagula61@gmail.com"
}
{
"eventType": "returned",
```

8.Database Design

DataBase name–naresh

```
CREATE TABLE users (
   id BIGINT AUTO_INCREMENT PRIMARY KEY,
   username VARCHAR(50) UNIQUE NOT NULL,
   password VARCHAR(255) NOT NULL,
   role VARCHAR(20) NOT NULL,
   name VARCHAR(100),
   department VARCHAR(100),
   email VARCHAR(150) NOT NULL
);

CREATE TABLE equipment (
   id BIGINT AUTO_INCREMENT PRIMARY KEY,
   name VARCHAR(100),
   type VARCHAR(50),
   status VARCHAR(20) DEFAULT 'available',
   CONSTRAINT uq_equipment_name_type UNIQUE (name, type)
);

CREATE TABLE allocations (
   id BIGINT AUTO_INCREMENT PRIMARY KEY,
   equipment_id BIGINT NOT NULL,
   user_id BIGINT NOT NULL,
   allocated_at DATETIME NOT NULL,
   expected_return DATETIME NOT NULL,
   returned_at DATETIME,
   equipment_condition VARCHAR(50),
   reminder_sent BOOLEAN DEFAULT FALSE NOT NULL,
   FOREIGN KEY (equipment_id) REFERENCES equipment(id),
   FOREIGN KEY (user_id) REFERENCES users(id),
   CONSTRAINT uq_active_allocation UNIQUE (equipment_id, returned_at)
);
```

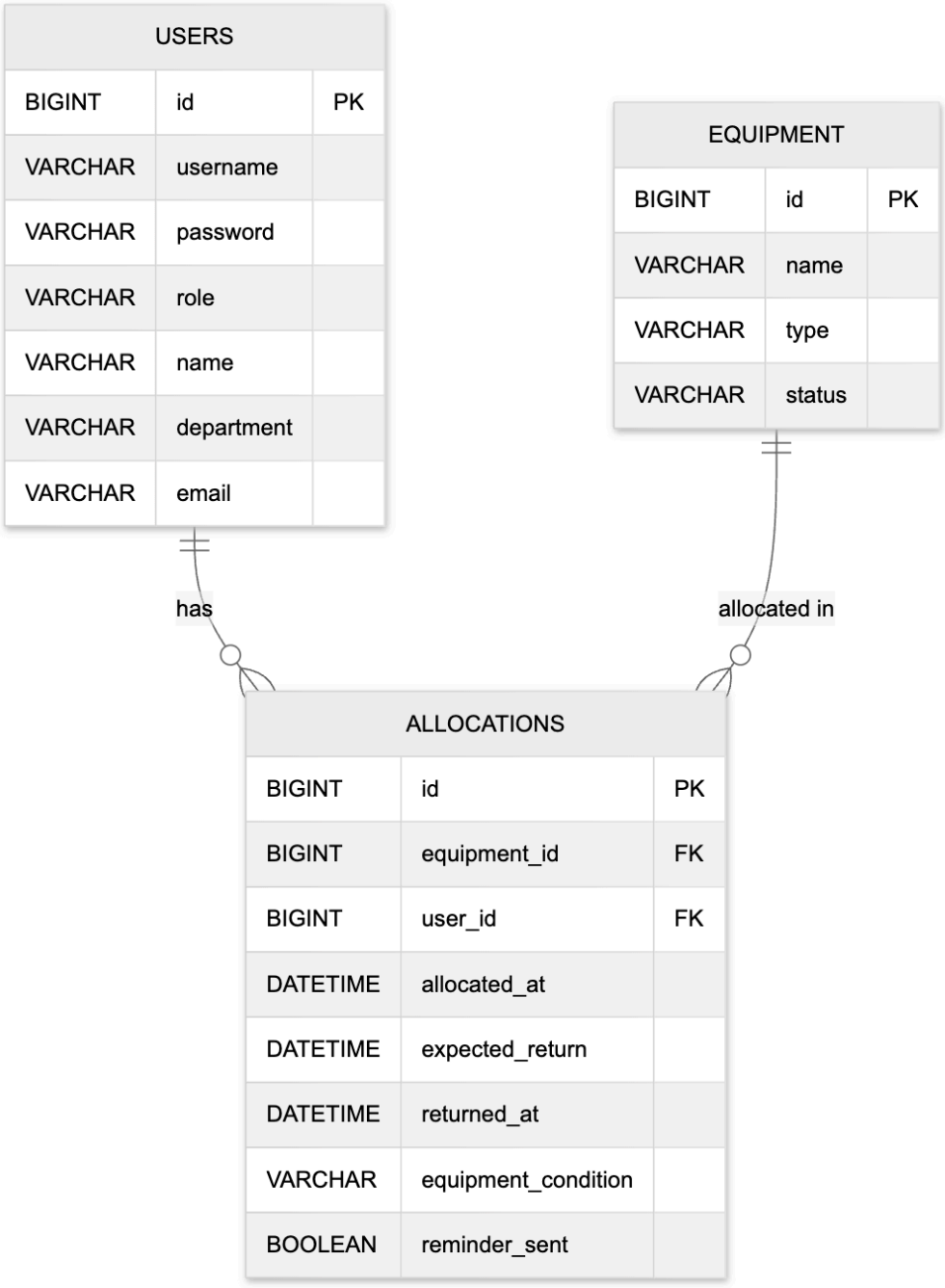
Insert Queries:

```
INSERT INTO users (id, username, password, role, name, department, email) VALUES
(1, 'admin', 'Admin@123', 'admin', 'Naresh Nagula', 'IT', 'naresh996463@gmail.com'),
(2, 'Hardhik', 'Hardhik@123', 'user', 'Hardhik', 'IT', 'naresh119935@gmail.com'),
(3, 'reception1', 'Recep@123', 'reception', 'Reception Staff', 'Front Desk',
'reception@example.com'),
(4, 'inventory1', 'Inv@123', 'inventory', 'Inventory Staff', 'IT', 'nareshnagula61@gmail.com'),
(5, 'maint1', 'Maint@123', 'maintenance', 'Maintenance Staff', 'Facilities',
'maintenance@example.com'),
(6, 'Rohith', 'Rohith@123', 'user', 'Rohith', 'IT', 'rohit123@gmail.com'),
(7, 'Kohli', 'Kohli@123', 'user', 'kohli', 'IT', 'kohli12@gmail.com'),
(8, 'Mahesh', 'Mahesh@123', 'user', 'mahesh', 'IT', 'mahesh2@gmail.com');


INSERT INTO equipment (id, name, type, status) VALUES
(1, 'hp laptop', 'Laptop', 'allocated'),
(2, 'apple laptop', 'Laptop', 'allocated'),
(3, 'Asus laptop', 'Laptop', 'allocated'),
(4, 'Apple laptop', 'Projector', 'allocated'),
(5, 'Mi', 'Projector', 'allocated'),
(6, 'Hp', 'Projector', 'damaged'),
(7, 'Dell', 'Laptop', 'available'),
(8, 'Intex', 'Laptop', 'available');


INSERT INTO allocations (id, equipment_id, user_id, allocated_at, expected_return, returned_at,
equipment_condition, reminder_sent) VALUES
(1, 1, 2, '2025-11-19 17:19:27', '2025-11-20 18:00:00', NULL, NULL, 1),
(2, 2, 6, '2025-11-19 17:27:17', '2025-11-20 18:00:00', NULL, NULL, 1),
(3, 3, 7, '2025-11-19 17:30:58', '2025-11-20 18:00:00', NULL, NULL, 1),
(4, 4, 8, '2025-11-19 17:39:59', '2025-11-20 18:00:00', NULL, NULL, 1),
(5, 5, 9, '2025-11-19 18:14:38', '2025-11-20 18:00:00', NULL, NULL, 1),
(6, 6, 10, '2025-11-19 18:19:40', '2025-11-20 18:00:00', '2025-11-19 22:42:59', 'Damaged', 0),
(7, 7, 11, '2025-11-19 18:58:12', '2025-11-20 06:42:23', NULL, 'Good', 1),
(8, 8, 10, '2025-11-19 21:14:27', '2025-11-20 18:00:00', '2025-11-19 22:15:35', 'Good', 0);
```

ERD DIAGRAM


USERS
BIGINT id PK
VARCHAR username
VARCHAR password
VARCHAR role
VARCHAR name
VARCHAR department
VARCHAR email
EQUIPMENT
BIGINT id PK
VARCHAR name
VARCHAR type
VARCHAR status
has
allocated in
ALLOCATIONS
BIGINT id PK
BIGINT equipment_id FK
BIGINT user_id FK
DATETIME allocated_at
DATETIME expected_return
DATETIME returned_at
VARCHAR equipment_condition
BOOLEAN reminder_sent

9. Security

Authentication: JWT-based tokens for secure, stateless user authentication.

Authorization: Role-based access control.

1. Admin
Responsibilities: Full control — manage users, equipment, allocation, returns, and mark equipment as damaged.

Flow:
User Management:

- Admin creates/updates/deletes users via /api/users/* endpoints.
- DB is updated immediately.

Equipment Management:

- Admin adds new equipment or updates details via /api/equipment/* endpoints.
- Status changes (e.g., damaged) are updated in DB.

Equipment Allocation/Return:

- Admin can allocate or return equipment via /api/equipment/allocate or /return.

Allocation/return triggers Kafka events:

- Notify Inventory Staff (availability)
- Notify Maintenance Staff if returned item is damaged

Mark Damaged:

- Admin can mark damaged equipment using /mark-damaged/:id.
- Publishes EquipmentDamagedEvent to Kafka → Maintenance Staff Actor handles it asynchronously.
- Summary: Admin is full control, performs actions synchronously in DB, events are sent asynchronously for notifications.

2. Reception Staff
Responsibilities: Handle day-to-day allocation and return of equipment.

Flow:

Equipment Allocation:

Receives allocation request → /allocate API called → DB updated.
Event published to Kafka → Inventory notified of decreased availability.

Equipment Return:

- Receives returned item → checks for damage:
- If OK → /return API called → DB updated → Inventory notified.
- If damaged → /mark-damaged/:id API called → DB updated → Maintenance notified via Kafka.

View Status:

- /status/:userId → view equipment allocated to specific user (read-only).
- Summary: Reception staff handles synchronous allocation/return and triggers asynchronous events for Inventory & Maintenance.

3. Inventory Staff

Responsibilities: Monitor inventory and availability.
Flow:
Receive Notifications:

- Kafka consumer listens for allocation/return events → updates internal inventory view.
- Damaged items are flagged as unavailable.

View Equipment & Status:

- /equipment/list → read-only list of equipment
- /status/:userId → read-only allocation status
- Summary: Inventory staff do not change DB, only monitor and plan inventory based on asynchronous event-driven updates.

4. Maintenance Staff

Responsibilities: Handle damaged equipment, ensure repair and readiness for allocation.
Flow:
Receive Alerts:

Kafka consumer listens for EquipmentDamagedEvent.
Maintenance staff inspects damaged items.

Repair/Update Status:

- After repair, update equipment status via API → "ready" or "repaired".
- Event published to Kafka → Inventory notified that equipment is now available.

View Allocation Status:

- /status/:userId → read-only for planning repairs.
- Summary: Maintenance staff react to asynchronous events, update status after servicing, ensuring inventory accuracy.

End-to-End Flow Example:
Allocation → Return → Damaged → Repair → Availability

1. Admin adds new equipment.
2. Reception staff allocates equipment to Employee A → DB updated, Kafka event → Inventory notified.
3. Employee returns equipment damaged → Reception calls markDamagedEquipment → DB updated, Kafka event → Maintenance notified.
4. Maintenance staff repairs laptop → updates status to "ready" → Kafka event → Inventory notified.
5. Inventory now sees equipment available → can be allocated again.

10. Key Design Decisions

- **Microservice Architecture:** Enables independent scaling of API, Notification Service, and database.
- **Kafka Messaging:** Ensures reliable, asynchronous notifications.
- **Akka Actors:** Efficiently handle scheduled and real-time notifications.
- **Dockerization:** Guarantees consistent deployment across environments.
- **Database Normalization:** Prevents duplication of equipment and employee data.
- **REST API:** Standardized communication between frontend and backend.

11. Conclusion

The proposed architecture provides a **scalable, reliable, and automated solution** for corporate equipment management. It ensures:

- Timely allocation and return of equipment.
- Automated notifications to employees and internal teams.
- Accurate tracking of inventory and maintenance needs.
- Maintainable and extensible system design for future expansion.